

Árvores binárias de busca

Assim como árvores binárias generalizam a ideia de listas encadeadas, árvores binárias de busca (= *binary search trees* = *BSTs*) generalizam a ideia de listas encadeadas crescentes.

Definição

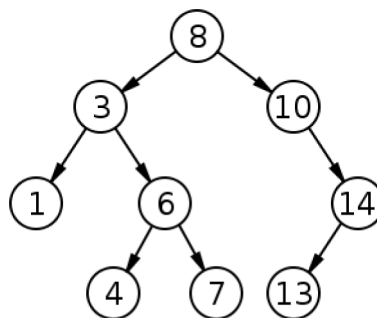
Considere uma [árvore binária](#) cujos nós têm um campo `chave` de um tipo linearmente ordenado, um tipo (como `int`, `char`, `string`, etc.) que admite comparação. Neste capítulo, suporemos que chaves são do tipo `int` e portanto os nós têm a seguinte estrutura:

```
typedef struct reg {
    int     chave;
    int     conteudo;
    struct reg *esq, *dir;
} noh; // nó
```

Uma árvore binária deste tipo é *de busca* se [cada nó](#) p tem a seguinte propriedade: ⚠ a chave de p é (1) maior ou igual à chave de qualquer nó na [subárvore](#) esquerda de p e (2) menor ou igual à chave de qualquer nó na subárvore direita de p . Em outras palavras, se p é um nó qualquer então

$$e \rightarrow \text{chave} \leq p \rightarrow \text{chave} \leq d \rightarrow \text{chave}$$

para todo nó e na subárvore esquerda de p e todo nó d na subárvore direita de p .



Eis outra maneira, talvez mais clara, de descrever a propriedade que define árvores de busca: a ordem [e-r-d](#) das chaves é crescente.

Exercícios 1

- ⚠ Suponha que $x \rightarrow \text{esq} \rightarrow \text{chave} \leq x \rightarrow \text{chave}$ para cada nó x dotado de filho esquerdo e $x \rightarrow \text{chave} \leq x \rightarrow \text{dir} \rightarrow \text{chave}$ para cada nó x dotado de filho direito. Essa árvore é de busca?
- IMPORTANTE! ⚠ Escreva uma função que decida se uma dada árvore binária é ou não é de busca.

Busca

Considere o seguinte problema: Dada uma árvore de busca r e um número k , encontrar um nó de r cuja chave seja k . A seguinte função recursiva resolve o problema:

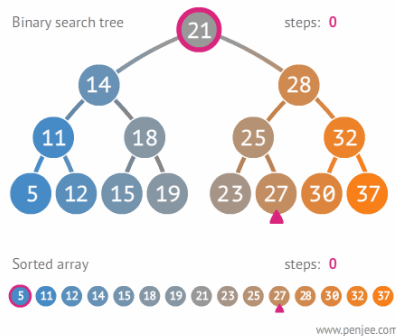
```
// Recebe uma árvore de busca r e
// um número k. Devolve um nó
// cuja chave é k; se tal nó não existe,
// devolve NULL.

noh *
busca (arvore r, int k) {
    if (r == NULL || r->chave == k)
        return r;
    if (r->chave > k)
        return busca (r->esq, k);
    else
        return busca (r->dir, k);
}
```

(O tipo-de-dados `arvore` é igual a um ponteiro-para-nó.) Eis uma versão iterativa da mesma função:

```
while (r != NULL && r->chave != k) {
    if (r->chave > k)
        r = r->esq;
    else
        r = r->dir;
}
return r;
```

No pior caso, a busca consome tempo proporcional à altura da árvore. Se a árvore for balanceada, o consumo será proporcional a $\log n$, sendo n o número de nós. Esse tempo é da mesma ordem que a busca binária num vetor ordenado.



Exercícios 2

1. Escreva uma função `min` que encontre uma chave mínima em uma árvore de busca. Escreva uma função `max` que encontre uma chave máxima.
2. Suponha que as chaves de nossa árvore de busca são distintas duas a duas. Escreva uma função que receba uma chave k e devolva a chave seguinte na ordem crescente.
3. Escreva uma função que transforme um vetor crescente em uma árvore binária de busca que seja balanceada.
4. Escreva uma função que transforme uma árvore de busca em um vetor crescente.
5. Há uma relação muito íntima entre árvores de busca e o algoritmo de busca binária num vetor crescente. Qual é, exatamente, essa relação?

Inserção

Considere o seguinte problema: Inserir um novo nó, com chave k , em uma árvore de busca. É claro que a árvore resultante deve também ser de busca. O novo nó tem a forma de uma folha avulsa e pode ser criado assim:

```
noh *novo;  
novo = malloc (sizeof (noh));  
novo->chave = k;  
novo->esq = novo->dir = NULL;
```

É claro que a função que resolve o problema deve devolver a raiz da nova árvore:

```
// A função recebe uma árvore de busca r  
// e uma folha avulsa novo e insere a folha  
// na árvore de modo que a árvore continue  
// sendo de busca. A função devolve a raiz  
// da árvore resultante.  
  
arvore  
insere (arvore r, noh *novo) {  
    if (r == NULL) return novo;  
    if (r->chave > novo->chave)  
        r->esq = insere (r->esq, novo);  
    else  
        r->dir = insere (r->dir, novo);  
    return r;  
}
```

(A raiz da árvore resultante é a mesma da árvore original a menos que a árvore original seja vazia.)

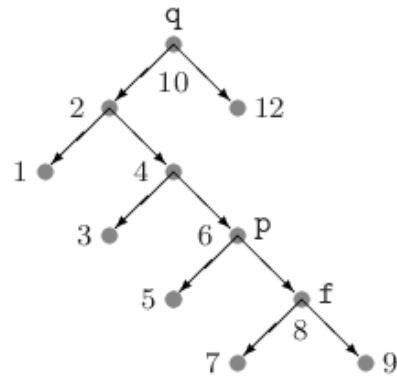
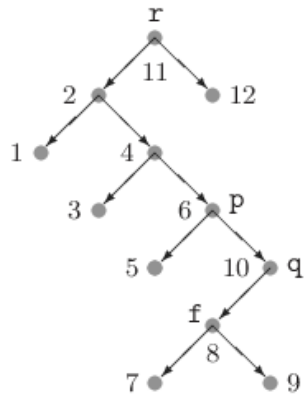
Exercícios 3

1. Que acontece se a função `insere` acima for invocada com `r` igual a `NULL`?
2.  Escreva uma versão não recursiva de `insere`.
3. Suponha que as chaves 50 30 70 20 40 60 80 15 25 35 45 36 são inseridas, nesta ordem, numa árvore de busca inicialmente vazia. Desenhe a árvore que resulta.
4. Suponha que os nós de uma árvore binária de busca têm um campo pai. Escreva um função que insira um novo nó na árvore. Faça duas versões: uma recursiva e uma iterativa.

Remoção

Problema: Remover um nó de uma árvore de busca de tal forma que a árvore continue sendo de busca.

Começamos tratando do caso em que o nó a ser removido é a raiz da árvore. Se a raiz não tem um dos filhos, basta que o outro filho assuma o papel de raiz. Senão, faça com que o nó anterior à raiz na ordem e-r-d assumo o papel de raiz.



Remoção de r . O nó anterior a r na ordem e-r-d é q (os nós estão numerados em ordem e-r-d). O nó q é colocado no lugar de r , os filhos de r passam a ser filhos de q , e f passa a ser filho (direito) de p .

```
// Recebe uma árvore não vazia r.
// Remove a raiz da árvore e rearranja
// a árvore de modo que ela continue sendo
// de busca. Devolve o endereço
// da nova raiz.
```

```
arvore removeraiz (arvore r) {
    noh *p, *q;
    if (r->esq == NULL) {
        q = r->dir;
        free (r);
        return q;
    }
    p = r; q = r->esq;
    while (q->dir != NULL) {
        p = q; q = q->dir;
    }
    // q é nó anterior a r na ordem e-r-d
    // p é pai de q
    if (p != r) {
        p->dir = q->esq;
        q->esq = r->esq;
    }
    q->dir = r->dir;
    free (r);
    return q;
}
```

Suponha agora que queremos remover um nó que não é a raiz da árvore. Para remover o filho esquerdo de um nó x faça

```
x->esq = removeraiz (x->esq);
```

e para remover o filho direito de x faça

```
x->dir = removeraiz (x->dir);
```

Exercícios 4

1. Suponha que as chaves 50 30 70 20 40 60 80 15 25 35 45 36 são inseridas, nesta ordem, numa árvore de busca inicialmente vazia. Desenhe a árvore que resulta. Em seguida remova o nó que contém 30.
2. Escreva uma versão recursiva da função que remove um nó de uma árvore de busca.

3. Suponha que os nós de uma árvore binária de busca têm um campo pai. Escreva um função que remova um nó da árvore. Faça duas versões: uma recursiva e uma iterativa.

O desempenho dos algoritmos

Quanto tempo gastam os algoritmos de busca, inserção e remoção discutidos acima? É claro que esse tempo é limitado pelo número de nós, digamos n , da árvore (pois nenhum nó é visitado mais que 3 vezes). Mas esta é uma resposta muito grosseira. Eis uma resposta mais fina: no pior caso, qualquer dos algoritmos acima

gasta uma quantidade de tempo proporcional à altura da árvore.

Conclusão: interessa trabalhar sempre com árvores balanceadas, ou seja, árvores que têm altura próxima a $\log n$, isto é, árvores em que todas as folhas têm aproximadamente a mesma profundidade.

Infelizmente não é fácil fazer isso. A altura da árvore é sujeita a chuvas e trovoadas. É muito fácil construir um exemplo em que uma árvore começa com altura próxima de $\log n$ mas depois de algumas inserções azaradas fica com altura muito mais próxima de $n-1$ (claro que o valor de n muda ao longo desse processo).

Os algoritmos de inserção e remoção descritos acima *não* produzem árvores balanceadas. Se a função `insere` for repetidamente aplicada a uma árvore balanceada, o resultado pode ser uma árvore bastante desbalanceada. Algo análogo pode acontecer depois de uma sequência de chamadas da função `remove`.

Para enfrentar essa situação é preciso inventar algoritmos bem mais sofisticados e complexos de inserção e remoção; esses algoritmos fazem um rebalanceamento da árvore após cada inserção e cada remoção. Vou apenas citar dois termos técnicos pitorescos: há um pacote de algoritmos de inserção e remoção que produz "árvores AVL"; há um outro pacote que produz "árvores rubro-negras".

Veja minha página [Árvores binárias de busca](#) baseada no [livro de Sedgewick e Wayne](#) (código Java)

Atualizado em 2016-03-15

<http://www.ime.usp.br/~pf/algoritmos/>

Paulo Feofiloff

[DCC-IME-USP](#)



